

Universidad Nacional
Autónoma de México
Facultad de Ingeniería
Fundamentos de VFX
Manual Tecnico
Lopez Lopez Axel Dion
319195576

1. Jugador

El módulo del jugador agrupa todos los sistemas relacionados con el personaje controlable: cámaras, movimiento, mecánicas activas y gestión de vida.

1.1 Control de Cámaras

Se utiliza la extensión Cinemachine para gestionar las dos cámaras disponibles en el juego:

- Cámara orbital: permite rotar la vista horizontalmente con el ratón; Cinemachine gestiona el movimiento.
- Cámara top-down: vista cenital fija.

El script CameraController se ocupa exclusivamente de alternar entre ambas cámaras al presionar Shift.

1.2 Movimiento

El movimiento del personaje es gestionado por el script playerMovement mediante el componente CharacterController de Unity. Las teclas A, W, S y D definen el vector de desplazamiento.

Con la cámara orbital activa, los componentes horizontales de la rotación de la cámara se aplican al vector de movimiento, lo que permite que el jugador se mueva siempre en la dirección visual esperada. La rotación del personaje se sincroniza con dicho vector mediante interpolación lineal esférica (Slerp), lo que genera un suavizado natural.

Al presionar la tecla de salto, el input de movimiento se ignora y el personaje avanza en su rotación actual (funcionamiento del dash).

1.3 Mecánicas Activas

Además del desplazamiento básico, el jugador dispone de cuatro mecánicas:

- Dash: al presionar el salto, el jugador se desplaza en la dirección actual y obtiene inmunidad al daño durante la acción. Cooldown: 0,5 segundos.
- Curación (tecla E): restaura el 50 % de la vida máxima sin superar el límite. Disponible 3 veces con un cooldown de 30 segundos entre usos.
- Escudo (clic derecho mantenido): activa una segunda hitbox frontal que rota junto al jugador. Bloquea proyectiles e impide recibir daño; el impacto de un proyectil empuja al jugador según la posición relativa del mismo. Mientras el escudo está activo, el movimiento se detiene, aunque la rotación sigue respondiendo a las teclas WASD y a la cámara.
- Ataque: genera una hitbox temporal frente al jugador y detiene el movimiento brevemente. Inflige daño al jefe si hay contacto.

1.4 Sistema de Vida

El jugador comienza con 100 puntos de vida. Cada golpe recibido aplica un daño específico y concede 0,5 segundos de invulnerabilidad. El dash ignora completamente las hurtboxes enemigas.

Si la vida llega a 0, o el jugador cae fuera de la arena, el juego se pausa y se muestra la pantalla de Game Over (la caída omite la vida actual).

2. Jefe

El jefe es el único enemigo del juego. Su comportamiento está regido por un ciclo de fases bien definido, con tres ataques distintos y un sistema de vida propio.

2.1 Ciclo de Ataque

El script bossAi ejecuta un ciclo continuo de tres fases:

- **Movimiento:** el jefe selecciona una posición dentro de la arena, en un aro alrededor del jugador, y se desplaza hasta ella. Al terminar, queda vulnerable a ataques.
- **Ataque:** ejecuta uno de tres ataques aleatorios, cada uno implementado en un script independiente que hereda de la interfaz BossAttack. Esta interfaz define ExecuteAttack (lógica principal) y CancelAttack (cancelación anticipada si el jefe recibe daño).
- **Espera:** el jefe permanece estático un breve periodo para dar oportunidad de atacar. Esta fase se omite si el jefe recibe daño.

2.2 Sistema de Vida

Gestionado por el script bossHealth. El jefe solo puede ser dañado por la hitbox del ataque del jugador. Cada impacto activa un toggle de invulnerabilidad que se mantiene hasta que el jefe completa de nuevo la fase de movimiento.

Cuando la vida llega a 0, el juego se pausa y se muestra la pantalla de victoria.

2.3 Hurtboxes y proyectiles

La lógica de daño reutilizable está repartida en tres scripts modulares:

- **hurtbox:** se asigna a un trigger de Unity. Al colisionar con la hitbox del jugador, llama a la función de daño de playerHealth. Dos toggles controlan su

comportamiento: `isActive` indica si puede infligir daño, y `destroyOnContact` desactiva el objeto al impactar al jugador o al escudo.

- `projectileMovement`: define la velocidad y el tiempo de vuelo del proyectil. Al agotar el tiempo, el objeto se desactiva (si `hurtbox` no lo hizo antes).
- `projectileSpawner`: gestiona la creación y reutilización de proyectiles mediante object pooling. Al iniciarse, genera un conjunto de copias desactivadas del prefab recibido. Al activarse, toma un proyectil libre de la pool, lo reposiciona, lo reactiva y se desactiva a sí mismo.

El spawner admite un campo opcional para un modelo visible durante el tiempo de espera previo al disparo, lo que facilita animar el proyectil antes de ser lanzado.

2.4 Ataques del Jefe

El jefe dispone de tres ataques seleccionados al azar:

- `Attack_starCone`: dispara 80 proyectiles en un arco de 45° con ángulos aleatorios, rotando el spawner antes de cada disparo. El spawner tiene tiempo de espera nulo y una pool de 100 objetos.
- `Attack_swordAttack1`: activa uno de cuatro spawners (ubicados alrededor del jugador) al azar, cada 2 segundos, un total de 5 veces. Cada spawner tiene un retraso de 1 segundo y un modelo vinculado que se vuelve visible durante ese intervalo.
- `Attack_slashAttack`: compuesto por dos scripts. El primero (`slashAttack`) rota la hitbox durante 0,8 s, la detiene 0,2 s y la activa 0,5 s antes de desactivarse, sincronizado con un sprite 2D animado. El segundo script instancia estas hitboxes en la posición del jugador a intervalos de 2 segundos, generando una adicional tras cada ataque. La primera hitbox tiene un ángulo aleatorio; las siguientes se desplazan en $360^\circ / (N \times 2)$ respecto a la anterior.

3. Interfaz Gráfica (UI) y Game Manager

Dos scripts coordinan la progresión del juego y la información mostrada al jugador.

3.1 GameManager

El script `GameManager` centraliza las funciones que los scripts de vida y la UI invocan para iniciar, pausar, reiniciar el juego y navegar entre menús:

- `StartMenu`: dos botones: iniciar partida y mostrar controles.
- `ControlsMenu`: muestra un resumen de los controles; hacer clic vuelve al menú inicial.

- Win / Game Over Menu: ambos comparten estructura: un botón para volver al menú principal y otro para reiniciar la partida.

3.2 Battle HUD

El script BattleHud se comunica con el resto del juego para actualizar en tiempo real los siguientes elementos:

- Barras de vida: la del jugador se sitúa a la izquierda con disposición vertical; la del jefe en la parte superior con disposición horizontal.
- Cooldown y cargas de curación: ubicados en la parte inferior, junto a la barra de vida del jugador. Muestra el cooldown activo y las cargas restantes mediante tres círculos verdes.
- Cooldown del dash: a la derecha de la pantalla, en paralelo con el indicador de curación.

4. Efectos Especiales

4.1 Shader del Océano Oscuro

Se utiliza un shader personalizado para cubrir el vacío del escenario con un "océano oscuro" animado. Su funcionamiento se divide en cuatro partes:

- UV pixelada: en lugar de usar el nodo UV directamente, se multiplica por una variable de resolución, se aplica el nodo Round y se divide de nuevo. Esto divide la textura en cuadrantes de color uniforme, generando el efecto de pixelado. A cada capa se le aplica movimiento y tiling mediante el nodo Tiling and Offset.
- Primera capa (ondas blancas): se aplica un mapa de ruido amplificado con el nodo Fraction para generar un efecto de ondas; el nodo Step filtra las ondas más delgadas y uniformiza el color a blanco sólido. El eje U avanza con el tiempo lineal y el eje V oscila con una función sinusoidal, simulando el movimiento del mar.
- Segunda capa (manchas moradas): generada a partir de un segundo mapa de ruido con Step y multiplicación de color. Solo tiene movimiento en el eje U, más lento que la primera capa, para crear sensación de profundidad.
- Capa de distorsión: antes de aplicar la UV pixelada a cada capa, se superpone un mapa de ruido con movimiento en el eje V, simulando la deformación del agua. También usa la UV pixelada para mantener la coherencia visual.

4.2 Animaciones, Partículas y Sonidos

- Cada script de comportamiento recibe referencias directas a los sistemas de partículas, clips de audio y el Animator del modelo que debe controlar. Las partículas

y los sonidos se activan directamente desde cada script; las animaciones se gestionan mediante las variables definidas en cada Animator.

Los modelos, texturas y partículas son de autoría propia, los sonidos usados fueron obtenidos de los siguientes enlaces

Slash - Rpg * Autor: colorsCrimsonTears

- [Enlace](#)

Slash1.ogg * Autor: wesleyextreme_gamer

- [Enlace](#)

Shooting Star 2 * Autor: plasterbrain

- [Enlace](#)

church-bell-5 * Autor: Spleencast

- [Enlace](#)

Click * Autor: budek

- [Enlace](#)

Cloud Poof * Autor: qubodup

- [Enlace](#)

Heal - Rpg * Autor: colorsCrimsonTears

- [Enlace](#)

Todos estos sonidos estan bajo la licencia creative commons

4.3 Ambientación

El skybox es completamente negro, complementado con niebla del mismo color y una luz azul oscura para reforzar la atmósfera. El jefe, partes del escenario y los ataques incluyen emisores de luz para contrastar con el entorno. Además, una fuente de luz está vinculada al jefe y se desplaza con él.

4.4 Outlines y Efecto de Afterimages

El jefe y sus espadas cuentan con un outline blanco que resalta sus texturas casi completamente negras, implementado con la extensión Quick Outline (Asset Store, Chris Nolet).

El efecto de afterimage es gestionado por el script `bossAfterImage`, que crea diez copias del modelo del jefe con un material semitransparente, aplicando la misma técnica de object pooling que los spawners de proyectiles. Durante la fase de movimiento, las copias se posicionan sucesivamente en el lugar del jefe y se desactivan tras un breve tiempo, generando la estela visual.